Formula 1 Podium Prediction

CSEN903 Advanced Computer Lab — Milestone 1 Report

Team members: *(add names and IDs)*

October 2026

Notebook: `notebooks/f1_podium_prediction.ipynb` Kaggle: *(add link)* GitHub: *(add link)*

Abstract

We predict, for every driver in every Formula 1 race from 1950 to 2024, whether the driver finishes on the podium (`positionOrder` ≤ 3), using only information available after qualifying and before the start. The raw relational tables are audited and cleaned on copies (160,076 `\N` sentinels converted, 85 duplicated driver-race pairs resolved, keys verified), three data-engineering questions are answered with multi-hop joins, and a leakage-safe modeling table of 24,724 starts is built. Feature groups are selected by adding one group at a time on a season-based validation split, which keeps the starting grid and the driver's championship standings after the previous round. A logistic regression, a random forest and a shallow neural network all beat a grid-only baseline: test PR-AUC 0.708, 0.718 and 0.699 against 0.647. Permutation importance, SHAP and LIME show that the models rely mostly on the starting grid; these explanations describe the models, not causes.

Contents

1 Task and dataset **3**

2 Exploratory data analysis before cleaning **3**
2.1 First look . . . 3
2.2 Where and when values are missing . . . 3
2.3 Key checks and the one-row-per-driver-per-race rule . . . 5

3 Cleaning **5**

4 Target, pre-race cutoff and metrics **7**

5 Data-engineering questions **7**
5.1 Q1: Which circuits convert a front-row start into a podium most often, and did that change after 2014? . . . 7
5.2 Q2: Do drivers podium more often at home, controlling for starting position? . . . 8
5.3 Q3: Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024? . . . 9

6 Modeling table and data split **10**

7 Feature engineering and selection **11**
7.1 Candidate features and their validity at the cutoff . . . 11
7.2 Exploratory evidence . . . 11
7.3 Empirical selection: adding one group at a time . . . 13

8 Preprocessing **13**

9 Models **14**

10 Results **15**

11 Experiments **16**
11.1 Feature-group ablation 16
11.2 Preprocessing-order experiments 17

12 Explainability (XAI) **17**
12.1 Global: permutation importance 18
12.2 Global: SHAP 18
12.3 Local: SHAP waterfall and LIME 19

13 Inference function **20**

14 Limitations **20**

15 Conclusion **21**

A Reproducibility and decision log **21**

1 Task and dataset

Task. The unit of analysis is one driver entering one race, identified by `(raceId, driverId)`. The target is binary: `podium = 1` if `positionOrder` ≤ 3, otherwise 0. The prediction is made **after qualifying and before the race starts**. Anything that is only known during or after the race (finishing position, points, laps, race time, fastest lap, retirement status, standings after the race) is *leakage* and is never used as an input.

Dataset. The Kaggle/Ergast *Formula 1 World Championship (1950–2024)* dataset: 14 relational CSV tables. `results` is the central table (one row per driver per race); its ID columns point to `races`, `drivers`, `constructors` (teams) and `status`, and `races` points to `circuits`. The raw files are never modified: every cleaning step works on in-memory copies.

Tools. Following the course labs: pandas, numpy, matplotlib, seaborn and missingno (Lab 1), scikit-learn (Lab 2), TensorFlow/Keras (Lab 3), shap and lime (Lab 4). The notebook runs top to bottom with a single *Run All*, on Kaggle or locally.

2 Exploratory data analysis before cleaning

2.1 First look

Each table was loaded with a plain `pd.read_csv`, letting pandas infer the column types. The Lab 1 routine (`head`, `info`, `describe`) on `results` immediately exposed the main data-quality issue: `position`, `milliseconds`, `fastestLap`, `rank` and `fastestLapSpeed` contain numbers but were read as text. The reason is the sentinel `\N`, which the dataset uses instead of an empty cell; `position` alone contains 10,953 of them. Because a sentinel is a real string, `isnull()` does not see it.

Table 1: Columns with the most `\N` sentinels. In total there are 160,076 sentinel cells in 30 columns, of which `isnull()` detects only 68 (empty q2/q3 strings).

Table	Column	Sentinels	Share of rows
results	time, milliseconds	19,079	71.3%
results	fastestLap, fastestLapTime, fastestLapSpeed	18,507	69.2%
results	rank	18,249	68.2%
constructor_results	status	12,608	99.9%
results	position	10,953	40.9%
qualifying	q3 / q2	6,819 / 4,625	65.0% / 44.1%
races	sprint and practice session dates/times	≈1,050–1,110	94–99%

2.2 Where and when values are missing

Figure 1 shows the missingness pattern of `results` and `qualifying` (with `\N` turned into NaN on a temporary copy), and Figure 2 shows how coverage changes over the 75 seasons.

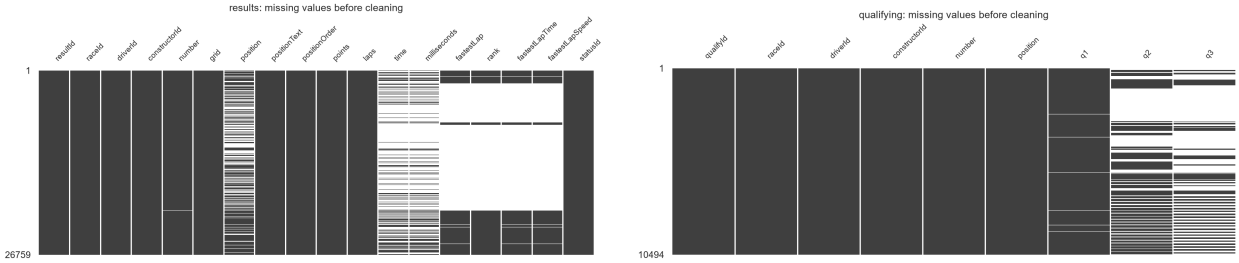


Figure 1: Missingness before cleaning (white = missing). In `results`, missing values are concentrated in the columns describing how the race ended.

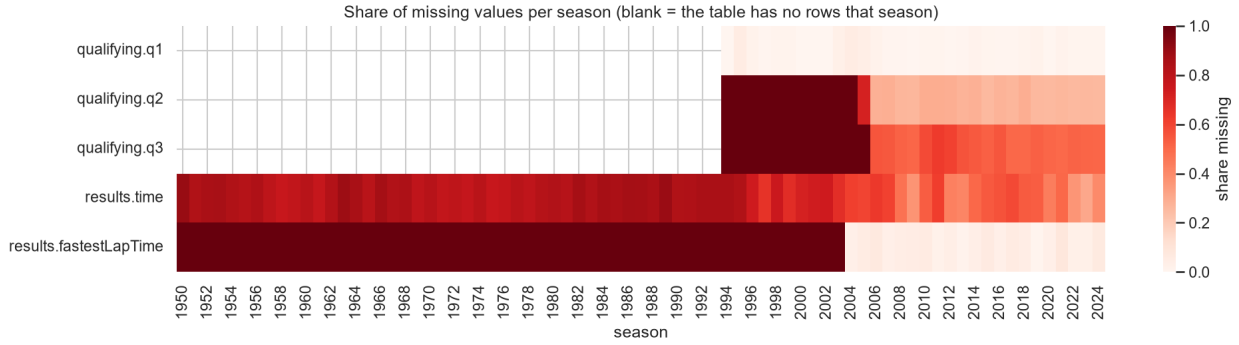


Figure 2: Share of missing values per season. The `qualifying` table starts in 1994; Q2/Q3 times are empty for every entry until 2003 and partly empty afterwards (only the faster drivers reach Q2/Q3: 28%/55% missing in 2014, 25%/51% in 2024). Fastest-lap times are missing for every season before the mid-2000s.

Findings that drive later decisions.

- A missing finishing `position` means the driver did not finish. This is information, not a gap, so it is never imputed.
- Qualifying times are missing for most of the history (**qualifying-coverage drift**). They therefore cannot be a feature over 1950–2024; the starting grid, which exists for every race, is used instead.
- `grid = 0` occurs 1,638 times and mixes two cases: 72 drivers who started from the pit lane (laps > 0) and drivers who never started (did not qualify or withdrew, 0 laps) (Figure 3).
- Before the 1990s many more drivers entered a race than could start, inflating the number of entries per race.
- The same value is spelled in different ways: `USA` / `United States` in `circuits`, and `Argentine` / `'Argentinian '` (with a trailing space), `East German` and double nationalities in `drivers`. These were found with `value_counts()`.

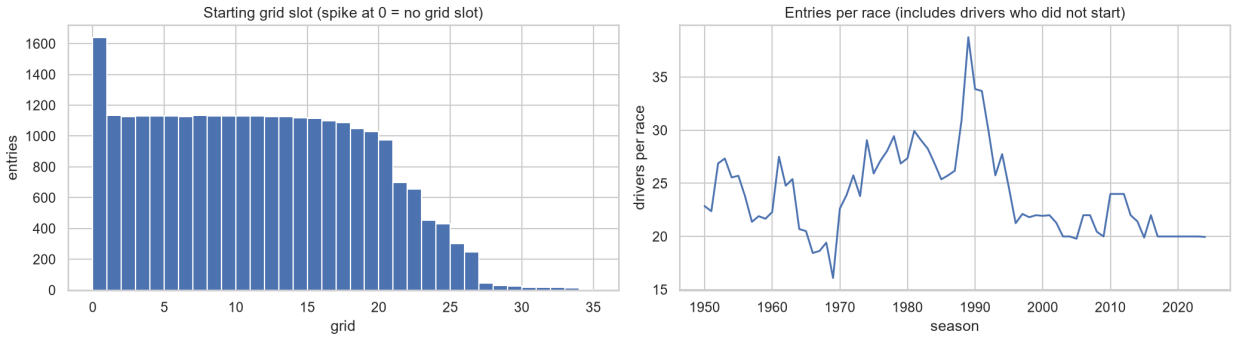


Figure 3: Left: starting grid slot, with a spike at 0 (no real slot). Right: entries per race, which include drivers who did not start.

2.3 Key checks and the one-row-per-driver-per-race rule

- **Primary keys.** All 14 primary keys (single columns, or `raceId+driverId+lap` for `lap_times` and `raceId+driverId+stop` for `pit_stops`) are unique and never empty.
- **Foreign keys.** All 22 foreign-key relations reconcile: every `raceId`, `driverId`, `constructorId`, `statusId` and `circuitId` exists in the table it points to (0 orphans). The notebook stops with an assertion if either check fails.
- **Duplicates.** 85 `(raceId, driverId)` pairs appear more than once (176 rows), all between 1950 and 1978. These are *shared drives*: a driver took over a teammate's car during the race.

Duplicate rule. We keep the row of the car the driver **started** in, meaning the row with the lowest real grid slot. Which car a driver starts in is known before the race; how each car finishes is not, so this rule matches the pre-race view of the task. If neither row has a grid slot, the driver never started; the row with the better finish is kept, and the row is excluded from modeling anyway. The alternative rule (keep the best finish) would change only 17 podium labels, all before 1979 (decision log, D3).

3 Cleaning

All cleaning is done on copies of the raw tables. Missing values are *not* filled during cleaning; filling happens later, using statistics from the training seasons only (Section 8).

Table 2: Cleaning steps and their effect.

Step	What it does and its effect
1. Sentinels	`\N` → NaN (160,076 cells); trim stray spaces; text columns whose values are all numbers become numeric (`results`: number, position, milliseconds, fastestLap, rank, fastestLapSpeed; `drivers.number`; three `sprint_results` columns).
2. Semantic typing	`races.date`, `drivers.dob` → datetime. Lap times and durations (`"1:27.452"`) → seconds for qualifying Q1–Q3, fastest laps, lap times and pit stops. The parsing matches the dataset's own millisecond columns exactly (589,081 lap times and all pit stops checked).
3. One spelling per value	`United States` → `USA`; `Argentinian` → `Argentine`; double nationalities → the first; `East German` → `German`. Every driver gets a `home_country` in the circuits' spelling (100% coverage asserted), used in Q2.
4. Unused columns	Web links and practice/qualifying/sprint session dates and times dropped.
5. One row per driver-race	Duplicate rule above: 26,759 → 26,668 rows; uniqueness asserted.
6. Flags	1,563 non-starters (grid 0 and 0 laps; excluded from modeling and Q3); 72 pit-lane starters (kept, placed one slot behind the last car on the grid); 381 Indianapolis 500 rows (1950–60, a US oval race rarely entered by F1 regulars; excluded from Q2 and modeling).
7. Status classes	Every finishing status is classed as finished (15,115), mechanical (6,770), accident (2,771), regulatory (1,944), driver (67) or other (1, "Refuelling"). Unclear reasons (punctures, wing failures, a plain "Retired") count as mechanical. Used in Q3.

Effect of the cleaning. After cleaning, the former sentinel columns of `results` are numeric (`str` → `float64`) and their missing values are visible to pandas (Figure 4). Removing non-starters brings the number of drivers per race down to the true starting fields, especially before 1990.

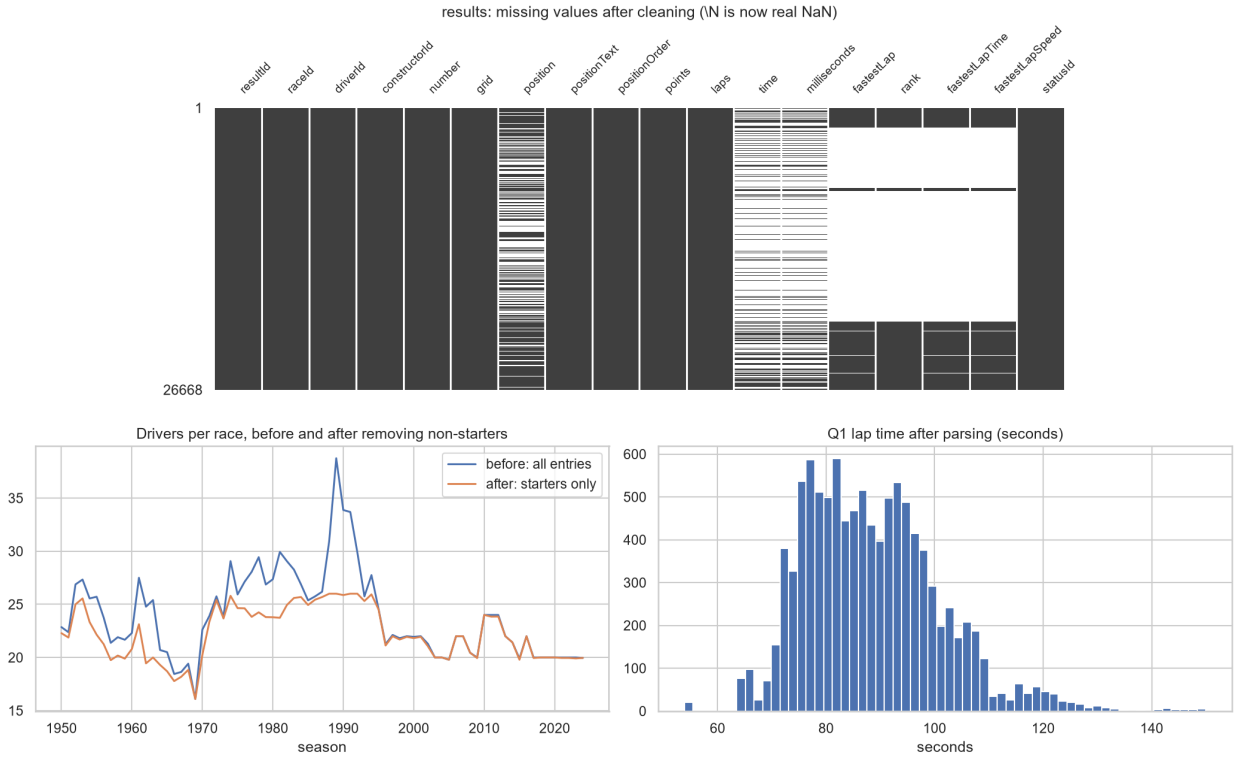


Figure 4: After cleaning. Top: `results` missingness with real NaN. Bottom left: drivers per race before and after removing non-starters. Bottom right: Q1 lap times after parsing to seconds (a plausible 60–120 s range).

4 Target, pre-race cutoff and metrics

Only drivers who started are counted. 13.5% of starts end on the podium (Figure 5). The podium rate follows 3/(number of starters), so it changes between eras with the field size.

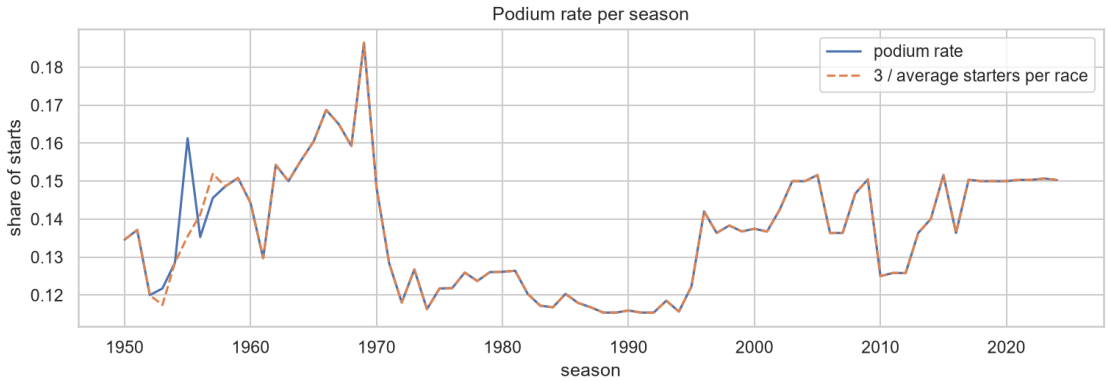


Figure 5: Podium rate per season and 3/average starters per race.

Table 3: The pre-race cutoff.

Allowed (known before the start)	Not allowed (only known after the race)

`grid` (qualifying result after penalties)
driver and team identity
championship standings **after the previous round**
 	
results of **earlier** races
	`position`, `positionText`, `positionOrder`, `points`
	`laps`, `time`, `milliseconds`, `statusId`
	`fastestLap`, `rank`, `fastestLapTime`, `fastestLapSpeed`
	this race's `lap_times`, `pit_stops`, and standings after this race

Sprint results are not used: sprints exist only since 2021.

Metric choice. With 13.5% positives, a model that always predicts "no podium" is about 86% accurate and useless, so accuracy is reported for reference only. The **primary metric is PR-AUC** (average precision): it measures how well the real podium finishers are ranked above everyone else, and it focuses on the rare class. The **secondary metrics** are ROC-AUC (overall separation of the two classes) and F1 at a decision threshold chosen on validation (the quality of the actual yes/no decision). Precision and recall are reported alongside F1.

5 Data-engineering questions

Each answer joins several tables. After every join the notebook asserts that the number of rows did not change, so no join duplicated rows. Rates are given with 95% Wilson intervals, which stay sensible for small samples.

5.1 Q1: Which circuits convert a front-row start into a podium most often, and did that change after 2014?

Method. Join `results` → `races` → `circuits`. **Grain:** one front-row starter (grid slot 1 or 2) in one race. **Qualifying vs. grid:** the two can differ because of grid penalties. We use `grid`, the slot the car actually started from: the podium is fought from there, and grid exists for every race since 1950, whereas qualifying positions start in 1994. **Missingness:** `grid` and `positionOrder` have no missing values. **Cutoff:** the front-row condition is known before the start; the podium

outcome is what is being measured. Circuits are ranked only if they have at least 10 front-row starts (5 races) in *each* era: a circuit used once has 2 front-row starts, so its rate can only be 0%, 50% or 100%. Sixteen circuits pass this threshold (21 at 5, 13 at 15, 7 at 20).

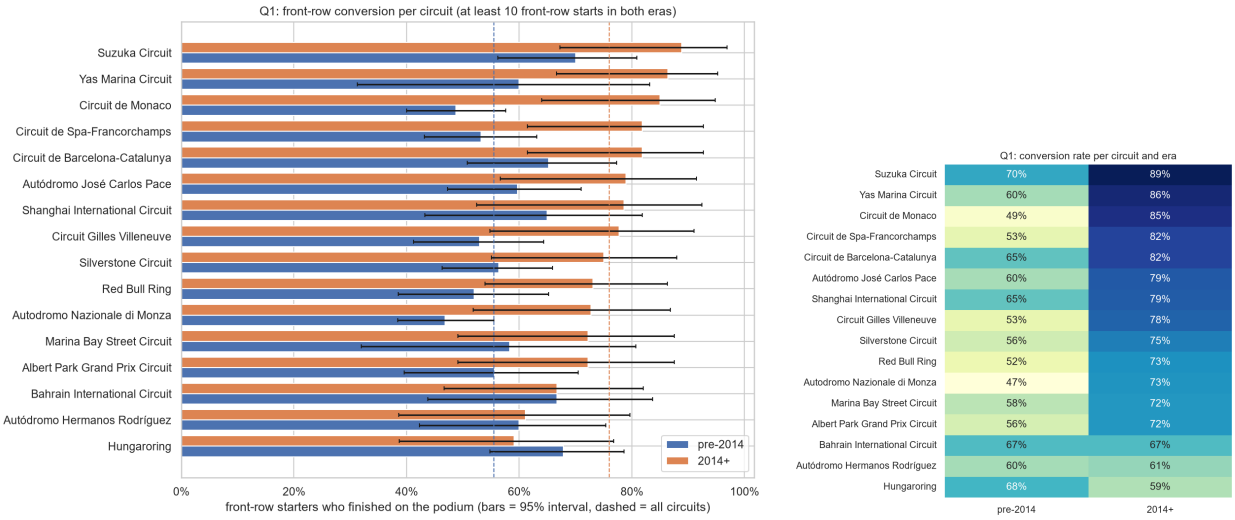


Figure 6: Q1: front-row conversion rate per circuit, before 2014 vs. 2014+ (bars = 95% intervals, dashed = all circuits), and the same values as a heatmap.

Answer. A front-row start converts much more often in the hybrid era: **55.5% before 2014** (1,002 of 1,805; 95% CI 53.2–57.8%) **vs. 76.0% from 2014** (345 of 454; 71.9–79.7%). The best circuits since 2014 are Suzuka (89%), Yas Marina (86%) and Monaco (85%); the weakest of the ranked circuits are the Hungaroring (59%) and Mexico City (61%). The ranking **changed completely**: the Spearman rank correlation between the two eras is −0.03. Monaco, for example, moved from 15th of 16 before 2014 (49%) to 3rd (85%). Which track rewards a front-row start depends on the era, not on the track alone.

5.2 Q2: Do drivers podium more often at home, controlling for starting position?

Method. Join `results` → `drivers` → `races` → `circuits`. A home race is a race in the driver's `home_country` (cleaning step 3). **Grain:** one start (non-starters and the Indianapolis 500 excluded): 24,724 starts, 1,944 of them at home (7.9%). **Missingness:** every driver has a home country (asserted). **Cutoff:** starting position and home status are known before the start. To control for starting position, podium rates are compared *within* starting-position groups (pole, 2–3, 4–6, 7–10, 11+), and the home-minus-away gaps are averaged, weighted by home starts. A logistic regression (Lab 2) of podium on home, starting group and era gives the home effect as an odds ratio.

Table 4: Q2: podium rate by starting group, home vs. away.

Start	Away rate	Home rate	Home – away	Home starts	Away starts
Pole	64.2%	58.3%	−5.9 pp	84	1,040
2–3	50.9%	53.7%	+2.9 pp	147	2,078
4–6	26.0%	26.4%	+0.4 pp	227	3,114
7–10	9.2%	7.1%	−2.1 pp	336	4,117
11+	1.6%	1.0%	−0.6 pp	1,150	12,431

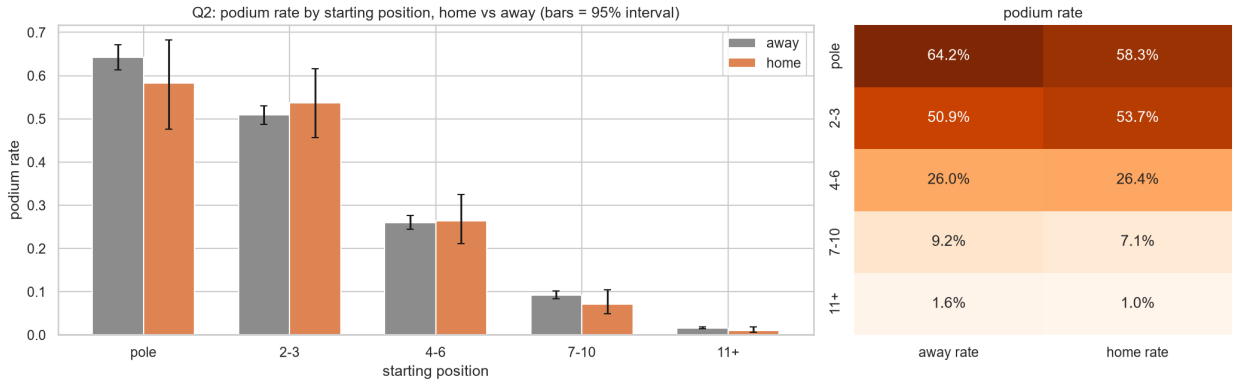


Figure 7: Q2: podium rate by starting position, home vs. away (bars = 95% intervals).

Answer. No home advantage. Starting from the same position, home drivers are 0.70 percentage points *less* likely to finish on the podium (weighted average), and the logistic regression gives an odds ratio of **0.915**, below 1 (1 = no effect). In every starting group the home and away 95% intervals overlap, so the small negative gap cannot be distinguished from no effect.

5.3 Q3: Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024?

Method. Join `results` → `status` (class from cleaning step 7) → `constructors`. **Grain:** one race start in 2014–2024 (4,619 starts). Non-starters are excluded from the denominator because they never raced. **Definition:** mechanical = the race ended because a car component failed; crashes, driver problems and regulatory decisions are excluded. Unclear reasons count as mechanical, which slightly raises all rates. **Missingness:** every observed status has a class (asserted). **Cutoff:** not relevant (a descriptive, post-race question). Teams are shown under their constructor name. The periods differ in length (8 vs. 3 seasons), so rates are compared, never counts. The smallest team sample is 31 starts (Marussia, 2014–2021).

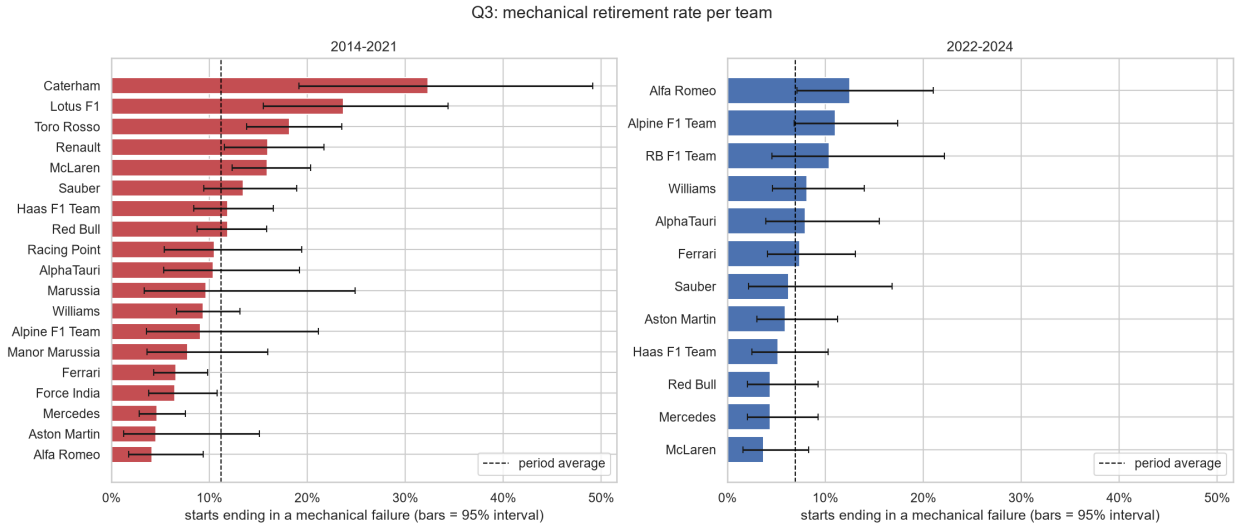


Figure 8: Q3: mechanical retirement rate per team and period (bars = 95% intervals, dashed = period average).

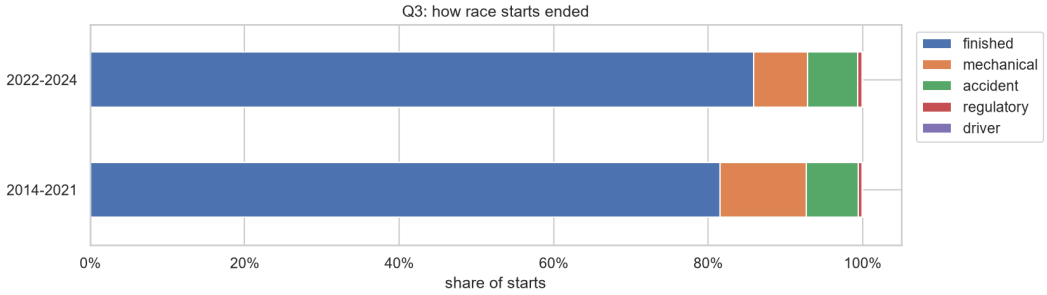


Figure 9: Q3: how race starts ended in each period.

Table 5: Q3: highest mechanical-retirement rates per period.

Period	Team	Starts	Mechanical	Rate
2014–2021	Caterham	34	11	32.4%
2014–2021	Lotus F1	76	18	23.7%
2014–2021	Toro Rosso	242	44	18.2%
2022–2024	Alfa Romeo	88	11	12.5%
2022–2024	Alpine F1 Team	136	15	11.0%
2022–2024	RB F1 Team	48	5	10.4%

Answer. Cars became more reliable: **11.2% of starts** (2014–2021; CI 10.1–12.3%) vs. **6.9%** (2022–2024; CI 5.7–8.4%) ended with a mechanical failure; the intervals do not overlap. The least reliable constructors were Caterham, Lotus F1 and Toro Rosso in 2014–2021, and Alfa Romeo, Alpine and RB in 2022–2024. Caterham's rate rests on only 34 starts, so its interval is wide.

6 Modeling table and data split

The modeling table has one row per driver who *started* a race in 1950–2024, without the Indianapolis 500: **24,724 rows**, with `(raceId, driverId)` unique (asserted). Pit-lane starters are placed one slot behind the last car.

Table 6: Season-based split.

Part	Seasons	Rows	Podiums	Rate	Used for
Train	1950–2017	21,749	2,895	13.3%	fitting models, imputer, scaler
Validation	2018–2021	1,619	243	15.0%	features, settings, thresholds
Test	2022–2024	1,356	204	15.0%	final scores, used once

Why split by season. The task is to predict future races. A random split (Lab 2's `train_test_split`) would put drivers from the same race on both sides, and the model would be trained on seasons later than the ones it is tested on. Both make scores look better than they are. Cutting by season mimics real use. The validation block (4 seasons, 243 podiums) is large enough to compare models, and the test block covers the newest regulations (2022–2024).

7 Feature engineering and selection

7.1 Candidate features and their validity at the cutoff

Table 7: Candidate features (11 in 5 groups). All use only information available before the race.

Group	Features	Why it is valid before the start
grid	`grid`	Set by qualifying and penalties before the start.
field size	`field_size`, `grid_rel`	Number of starters and grid ÷ starters; the starting field is known at the start.
driver standings	`drv_pos`, `drv_pts_share`, `drv_wins`	Championship position, points ÷ leader's points, and wins, taken from the standings **after the previous round**. Round 1 has no previous round, so the values are missing there.
team standings	`con_pos`, `con_pts_share`, `con_wins`	The same for the constructor, after the previous round.
recent form	`drv_podium_last5`, `con_podium_last5`	Podium share in the previous 5 races of the driver (team: share of its cars), computed with a shift so the current race is excluded.

Points are used as a share of the leader's points so that seasons with different points systems are comparable. Driver and team identity enter only through these features. One-hot encoding the ≈900 driver IDs would add hundreds of sparse columns, and many test drivers never appear in training. **Leakage check:** the notebook recomputes recent form for 150 random rows by hand, from earlier races only, and asserts that the values match. Standings are joined on the previous round's `raceId`, with a check that the join adds no rows.

7.2 Exploratory evidence

For each candidate, the training rows are split into five equal-sized groups by value, and the podium rate of each group is plotted (Figure 10); Spearman correlations are shown in Figure 11. Spearman is used instead of Pearson because the relations are monotonic but not linear: for `grid`, the podium rate drops steeply over the first slots and then flattens.

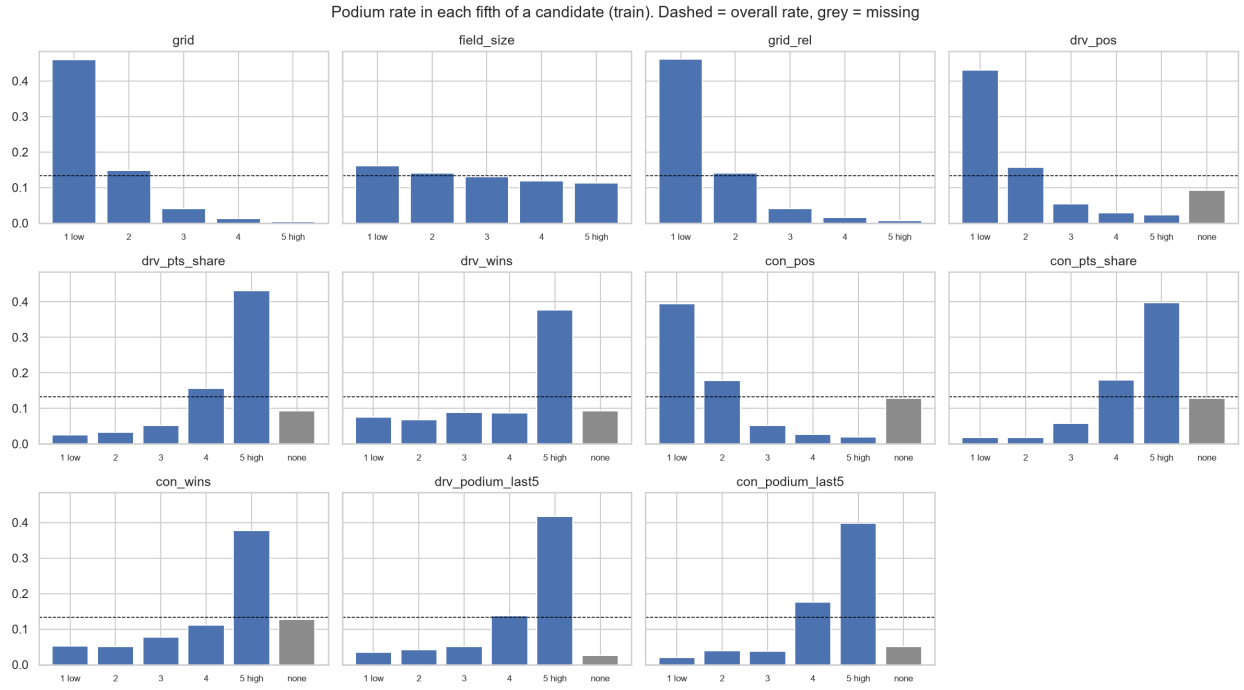


Figure 10: Podium rate in each fifth of every candidate (training seasons). Dashed = overall rate; grey = missing value.

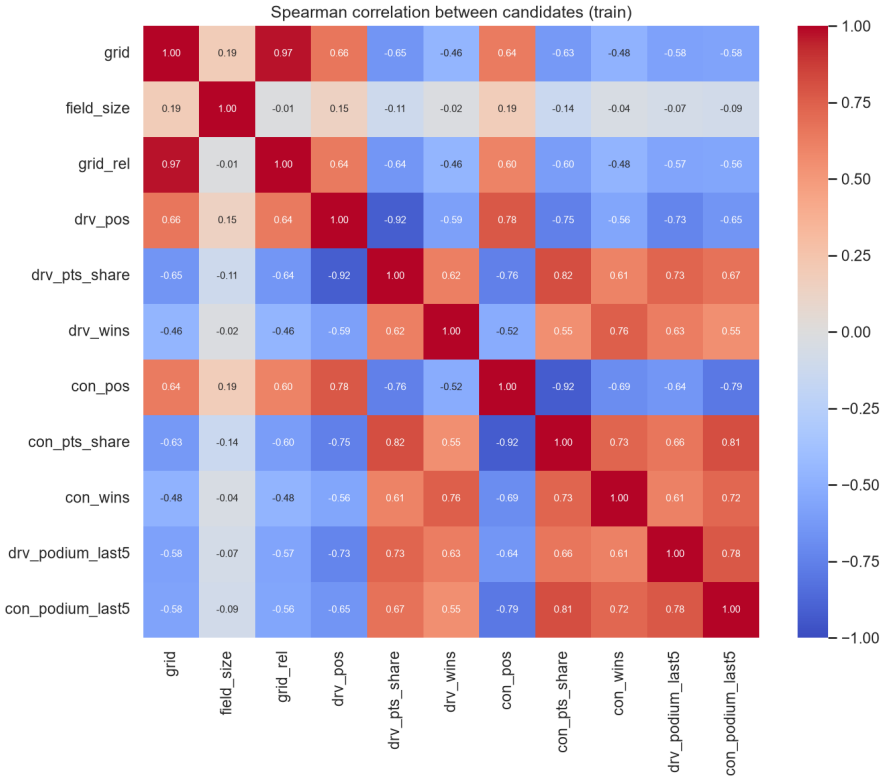


Figure 11: Spearman correlation between candidates (training seasons).

Correlation with the target (Spearman, train): `grid` −0.449, `grid_rel` −0.444, `drv_podium_last5` 0.434, `con_podium_last5` 0.414, `drv_wins` 0.404, `con_wins` 0.402, `drv_pts_share` 0.400, `drv_pos` −0.400, `con_pts_share` 0.395, `con_pos` −0.385, `field_size` −0.053. Most candidates are individually strong, but many repeat each other (grid and relative grid, position and points share, driver and team standings, form and standings). The exploratory

plots therefore cannot decide the final list on their own.

7.3 Empirical selection: adding one group at a time

Starting from `grid` only, one group is added at a time. Logistic regression and a random forest are retrained with fixed settings so that only the features change. **A group is kept if validation PR-AUC rises by at least 0.005 for both models**; smaller changes are within the noise of 243 validation podiums.

Table 8: Feature-selection trials (validation PR-AUC).

Step	LR	RF	Decision
grid only (baseline)	0.668	0.668	start
+ field size	0.669	0.671	dropped (gain < 0.005)
+ driver standings	0.714	0.726	**kept**
+ team standings	0.705	0.703	dropped (worse)
+ recent form	0.703	0.724	dropped (worse)
all 11 candidates together	0.702	0.720	for comparison

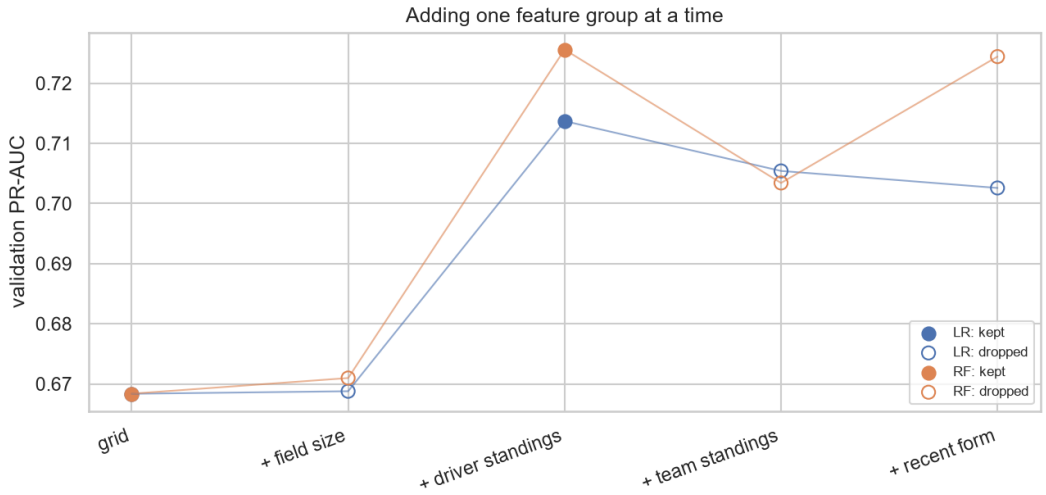


Figure 12: Validation PR-AUC as groups are added (filled = kept, empty = dropped).

Selected features (4): `grid`, `drv_pos`, `drv_pts_share`, `drv_wins`. The driver standings are the only group that adds on top of the grid: how the driver is doing *this season* is the strongest addition to where they start. Team standings and recent form make both models worse because they largely repeat the driver standings. Using all 11 candidates at once also scores lower than the selected set. A limitation of this method is that the result can depend on the order of the groups, and a dropped group is not retried; the all-candidates comparison and the ablation (Section 11.1) are checks on this.

8 Preprocessing

1. **Imputation (Lab 1):** missing values (round 1 standings, rookies) are filled with the **median** of the training rows (`SimpleImputer`). The median is robust to skewed columns such as wins. Fill values: grid 12, `drv_pos` 12, `drv_pts_share` 0.077, `drv_wins` 0.
2. **Scaling (Lab 3):** `StandardScaler` gives each feature mean 0 and standard deviation 1, so large-valued features (grid 1–34, positions up to 102) do not dominate small ones (shares

0–1). Scaling matters for logistic regression and the neural network; the forest is unaffected, but every model receives identical inputs so the comparison is fair.

Both steps are fit on the **training seasons only** and then applied unchanged to validation and test. Figure 13 shows the effect: scaling shifts and stretches the values but keeps the shape of the distribution (for example the spike at `drv_pts_share` = 0, when drivers have no points yet). Training data ends up with mean 0 and std 1; test does not (means between −0.27 and +0.23), which confirms the scaler was not refit on it.

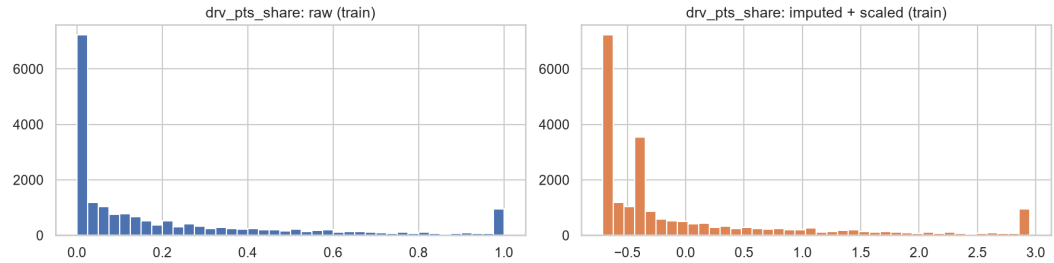


Figure 13: `drv_pts_share` before and after imputation and scaling (training rows).

9 Models

All models use the same rows and the same four inputs, so their scores are directly comparable.

- **Baseline: logistic regression on `grid` only.** The simplest reasonable model. Any other model must beat it to justify its extra complexity.
- **Logistic regression** (Lab 2): a weighted sum of the inputs passed through a sigmoid, giving a probability. It is fast and interpretable, but it can only learn effects that keep going in one direction.
- **Random forest** (Lab 4): 300 decision trees, each trained on a random sample of rows and features; their votes are averaged. It learns curved effects and interactions, but it is harder to interpret and its probabilities are coarse.
- **Shallow neural network** (Lab 3, Keras): one or two hidden `Dense` ReLU layers and a single sigmoid output neuron, trained with binary cross-entropy and Adam. Early stopping on validation loss (patience 15, best weights restored) guards against overfitting. It is flexible but a black box, and it depends on scaling and random initialisation.

Tuning (validation only). Each combination of class weighting (none / `balanced`) and model settings was trained, and the one with the best validation PR-AUC was kept (Table 9). The decision threshold of each model is the one that maximises F1 on validation: baseline 0.427, logistic regression 0.399, random forest 0.431, network 0.329.

Table 9: Tuning results (validation PR-AUC). Best settings in bold.

Model	Settings	No class weight	Balanced
LR	$C = 0.01$ / 0.1 / 1 / 10	0.711 / 0.714 / 0.714 / **0.714**	0.711 / 0.711 / 0.711 / 0.711
RF	depth 4, min leaf 20 / 50	0.723 / 0.715	0.717 / 0.717
RF	depth 6, min leaf 20 / 50	**0.726** / 0.719	0.723 / 0.723
RF	depth 8, min leaf 20 / 50	0.720 / 0.719	0.718 / 0.722
NN	hidden (16) / (32) / (32→16)	**0.727** / 0.725 / 0.726	–

Class weighting never helped. The regularisation strength C barely matters for logistic regression. The three network sizes score within 0.003 of each other, so the smallest one (one hidden layer of 16

units) is kept: with four inputs a bigger network gains nothing. It trained for the full 200 epochs because the validation loss was still decreasing slowly (Figure 14).

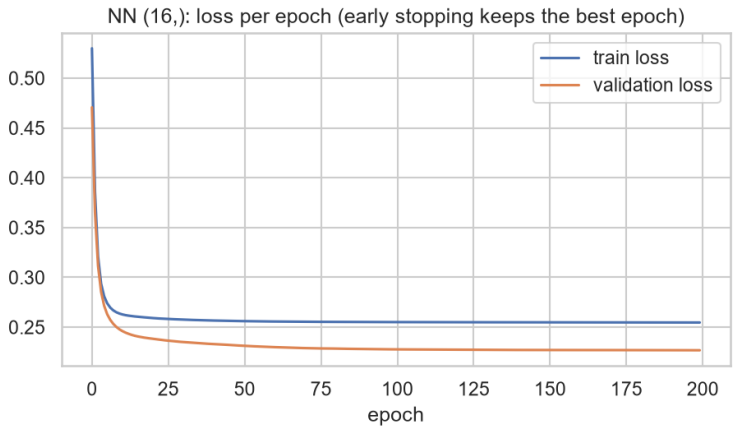


Figure 14: Neural network: training and validation loss per epoch.

10 Results

Table 10: All metrics on train, validation and the unseen test set. Thresholds were chosen on validation.

Model	Set	PR-AUC	ROC-AUC	F1	Precision	Recall	Accuracy
Baseline (LR on grid)	train	0.487	0.881	0.534	0.533	0.535	0.876
	val	0.668	0.918	0.712	0.712	0.712	0.914
	test	0.647	0.914	0.649	0.650	0.647	0.895
Logistic regression	train	0.545	0.891	0.535	0.565	0.507	0.882
	val	0.714	0.928	0.709	0.702	0.716	0.912
	test	0.708	0.930	0.678	0.690	0.667	0.905
Random forest	train	0.567	0.900	0.545	0.587	0.508	0.887
	val	0.726	**0.939**	**0.724**	0.715	0.733	0.916
	test	**0.718**	**0.940**	0.667	0.711	0.627	0.906
Neural network	train	0.550	0.895	0.563	0.516	0.620	0.872
	val	**0.727**	0.936	0.720	0.659	0.794	0.907
	test	0.699	0.938	**0.683**	0.610	0.775	0.892

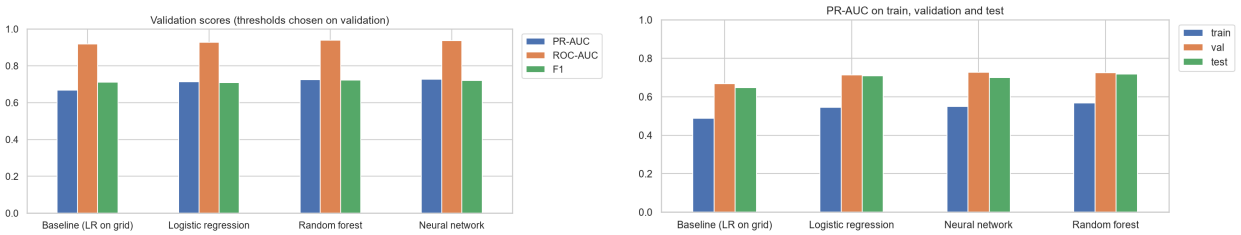


Figure 15: Left: validation PR-AUC, ROC-AUC and F1. Right: PR-AUC on train, validation and test.

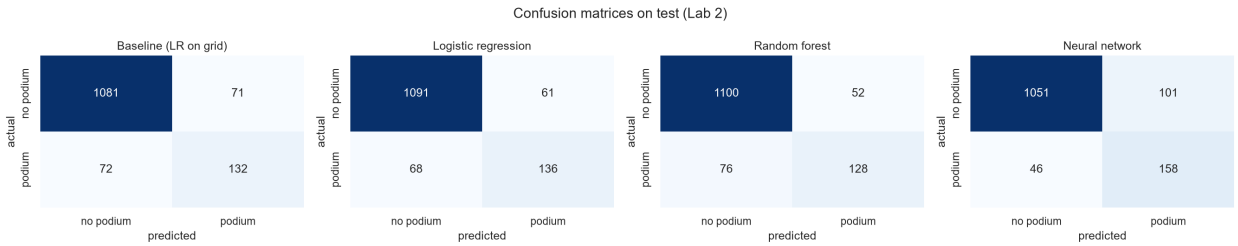


Figure 16: Confusion matrices on the test seasons (204 podiums among 1,356 starts).

Interpretation.

- **All three models beat the baseline** on validation and test. On test, PR-AUC rises from 0.647 to 0.699–0.718 and ROC-AUC from 0.914 to 0.930–0.940. Adding the driver's season standings to the grid is worth the extra complexity.
- **The three models perform about the same.** On validation the network and the forest are tied (0.727 vs. 0.726); on test the forest is best (0.718). Differences of 0.01–0.02 are within the noise of 204–243 podiums.
- **Final model: the neural network**, because it has the best *validation* PR-AUC, the primary metric. Choosing the forest because of its test score would use the test set for model selection, which the brief forbids. The network also has the highest test F1 (0.683) and recall (finds 158 of 204 podiums), at the cost of more false alarms (101), while the forest is more conservative (128 found, 52 false alarms).
- **No overfitting.** Train scores are *lower* than validation (PR-AUC 0.55–0.57 vs. 0.71–0.73). This is not underfitting: the training seasons include the 1950s–80s, with bigger fields, far more mechanical retirements and less predictable races. Test scores are within 0.03 of validation.

11 Experiments

11.1 Feature-group ablation

Each kept group is removed in turn, and logistic regression and the random forest are retrained with their chosen settings.

Table 11: Drop in PR-AUC when a feature group is removed.

Removed group	Model	Validation drop	Test drop
grid	Logistic regression	0.078	0.143
grid	Random forest	0.095	0.159
driver standings	Logistic regression	0.045	0.061
driver standings	Random forest	0.057	0.070

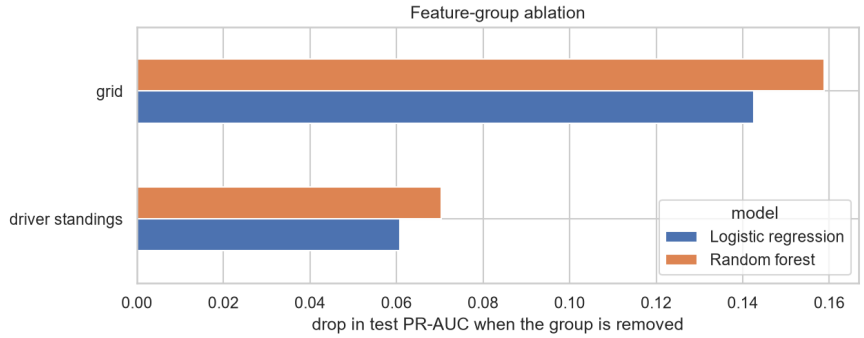


Figure 17: Feature-group ablation (drop in test PR-AUC).

The grid is the most important group (a drop of 0.14–0.16 on test). The driver standings add 0.06–0.07 on test and 0.05–0.06 on validation. Both kept groups contribute on validation *and* on test, so the selection made before training holds up after training.

11.2 Preprocessing-order experiments

Only the preprocessing changes; the models and their settings are fixed. Scores are on validation.

Table 12: Preprocessing-order experiments (validation PR-AUC). p-values from a paired bootstrap over 2,000 redraws of the validation races.

Experiment	Model	Used order	Alternative	Difference	p
1. Fit imputer + scaler on train only	LR	0.7138	0.7137	−0.0001	0.69
vs. on all seasons (leaky)	RF	0.7256	0.7252	−0.0004	0.29
2. Impute → scale	LR	0.7138	0.7138	+0.0000	0.99
vs. scale → impute	RF	0.7256	0.7256	0.0000	1.00

Experiment 1 fits the imputer and scaler on all seasons, including validation and test, before splitting. The test seasons then influence how the training data is transformed, which is leakage. The effect here is tiny and not significant, because 88% of all rows are training rows, so the medians, means and standard deviations hardly move. Fitting on train only remains the correct order: it is the only one possible when predicting a real future race.

Experiment 2 swaps the order of imputation and scaling. The filled value is the same point either way; only the scaler's mean and standard deviation differ slightly, with no measurable effect. Imputing first is kept, so the scaler sees exactly the complete data the model receives.

12 Explainability (XAI)

Explanations are not causal effects. A feature being important to a model means the model *relies on it to make predictions*. It is not evidence that the feature *causes* podium finishes. For example, a front grid slot is mostly a sign of a fast car, and the fast car is what produces podiums. Correlated inputs (the three standings columns) share the credit, and how they share it depends on the method.

The explanations use the **validation rows** (2018–2021), so the test set stays unseen until final scoring. Inputs are shown in their original units (the scaling is undone for display only). The detailed SHAP and LIME plots use the **random forest**: it is tied for the best validation PR-AUC, and `TreeExplainer` (Lab 4) computes exact SHAP values for trees.

12.1 Global: permutation importance

One column of the validation rows is shuffled, which breaks its link to the target while keeping its distribution. The trained model is then rescored without retraining. The drop in validation PR-AUC, averaged over 10 shuffles, measures how much the model relies on that column.

Table 13: Permutation importance: mean drop in validation PR-AUC.

Feature	Baseline	Logistic regression	Random forest	Neural network
`grid`	0.516	0.414	0.373	0.362
`drv_pos`	–	0.003	0.025	0.015
`drv_pts_share`	–	0.029	0.018	0.014
`drv_wins`	–	0.010	0.006	0.053

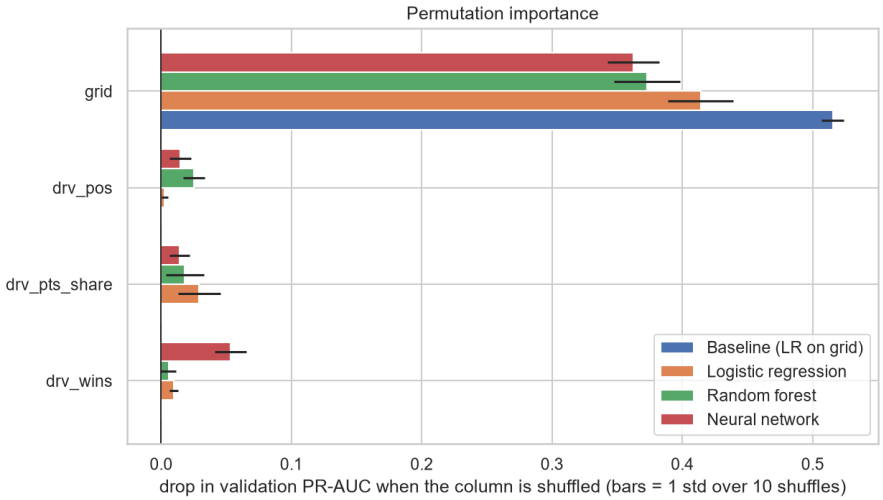


Figure 18: Permutation importance for all models (bars = 1 standard deviation over 10 shuffles).

Interpretation. The grid dominates for every model. Each model relies on a *different* standings column: the forest on `drv_pos`, logistic regression on `drv_pts_share`, and the network on `drv_wins`. These columns all describe how the driver is doing this season, so a model can take that information from any of them. For the same reason, permutation importance understates correlated inputs: "small" means "adds little beyond the other inputs", not "carries no information".

12.2 Global: SHAP

For each prediction, SHAP splits *prediction* − *average prediction* into one contribution per input; the contributions add up exactly to the model's output (verified in the notebook). The base value (average predicted probability) is 0.133.

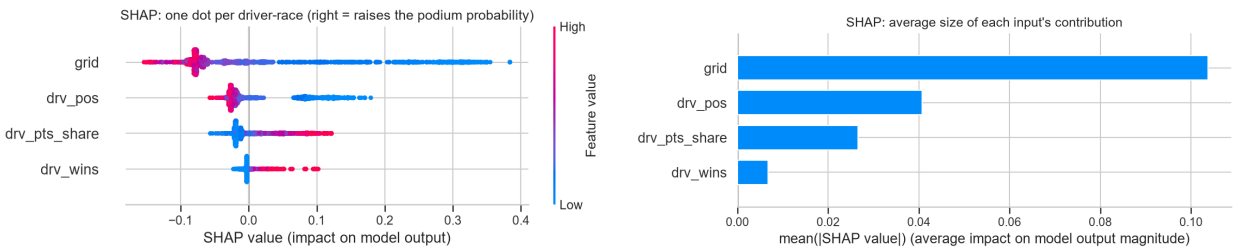


Figure 19: SHAP for the random forest on validation rows. Left: one dot per driver-race (colour = feature value; right = raises the podium probability). Right: mean |SHAP| per feature.

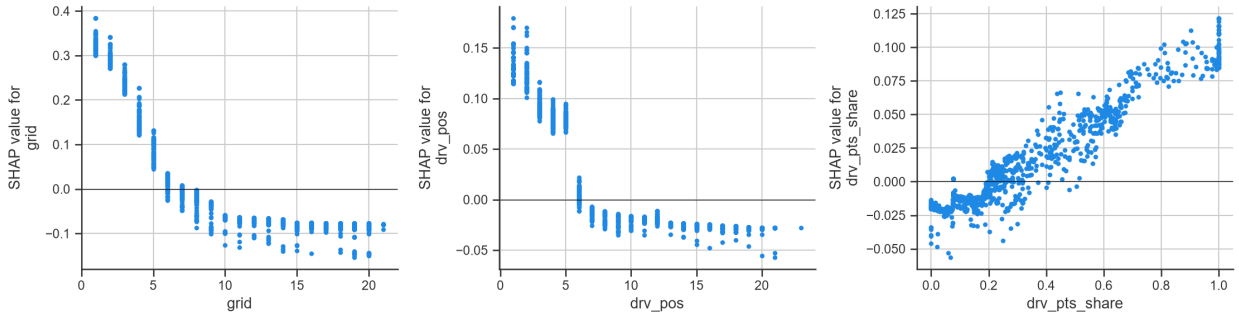
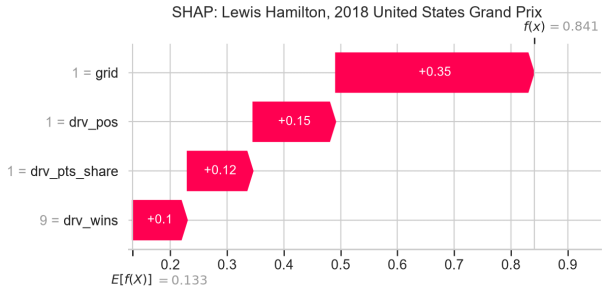


Figure 20: SHAP dependence plots for the three most important features.

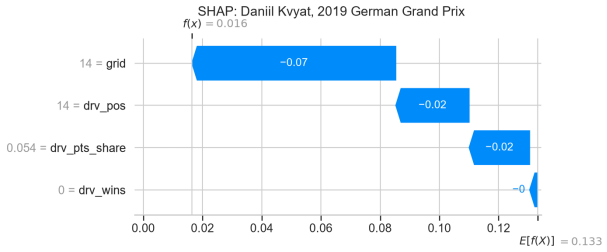
Interpretation. The grid is first by far (mean |SHAP| 0.104, about 2.5 times `drv_pos` at 0.041; then `drv_pts_share` 0.027 and `drv_wins` 0.007). Front grid slots push the probability up by as much as +0.35. The dependence plots show the shape: the grid contribution falls steeply from pole ($\approx +0.33$) to 5th ($\approx +0.08$), crosses zero around 6th–8th, and is flat (about -0.08) from 10th back, so from the second half of the grid the exact slot barely changes the prediction. Championship position acts as a step: the top five add +0.07 to +0.18, and from 6th back the effect is small and flat. The points share raises the probability steadily, from about -0.02 with no points to +0.10 for the leader. SHAP and permutation importance rank the forest's features in the same order, although they measure different things: lost ranking quality vs. how far an input moves the probability.

12.3 Local: SHAP waterfall and LIME

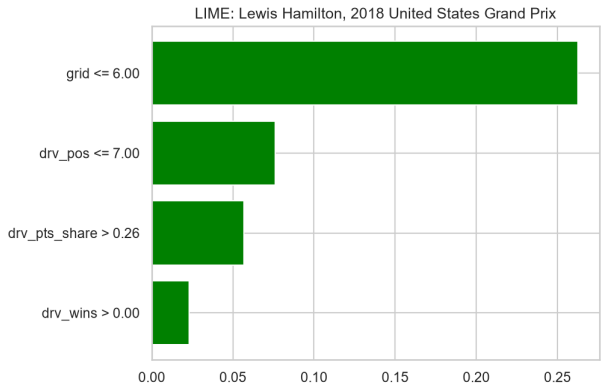
Two validation rows were selected by rule rather than by hand: the true podium finisher the forest was most sure about (*hit*) and the one it was least sure about (*miss*). LIME (Lab 4) perturbs the row many times, asks the forest about each copy, and fits a small linear model around the row.



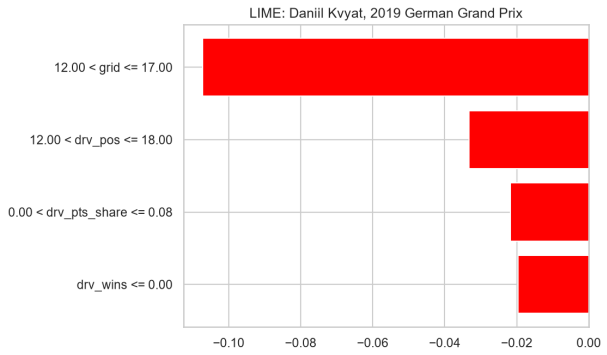
(a) SHAP, hit



(b) SHAP, miss



(c) LIME, hit



(d) LIME, miss

Figure 21: Local explanations for one confident hit and one miss.

Table 14: SHAP contributions and LIME weights for the two rows.

	Hit: Hamilton, USA 2018			Miss: Kvyat, Germany 2019		
Feature	Value	SHAP	LIME	Value	SHAP	LIME
`grid`	1	+0.350	+0.263	14	−0.069	−0.107
`drv_pos`	1	+0.146	+0.076	14	−0.025	−0.033
`drv_pts_share`	1.00	+0.116	+0.057	0.054	−0.020	−0.022
`drv_wins`	9	+0.097	+0.023	0	−0.003	−0.020
Predicted P(podium)	0.84 (finished on the podium)			0.02 (finished 3rd)		
LIME local fit R^2	0.82			0.15		

Interpretation. For the *hit* (Lewis Hamilton, 2018 United States GP, from pole as championship leader), every input pushes the probability up from the 0.133 base, the grid most of all. LIME agrees on every direction and on the ranking, and its local model fits well ($R^2 = 0.82$). For the *miss* (Daniil Kvyat, 2019 German GP, started 14th and finished 3rd), every input pushes down. Given what was known before the start, the forest's answer is reasonable: that race was wet and chaotic, an event that pre-race information cannot contain. LIME agrees on every direction, but its local model fits poorly here ($R^2 = 0.15$), so for this row the SHAP values are more trustworthy. SHAP divides the difference from the average prediction exactly among the inputs, while LIME's weights come from an approximate local linear model and need not add up.

13 Inference function

`predict_podium(record)` takes one driver-race as a dictionary of pre-race features in their normal units. Missing or unknown values are allowed: they are filled with the training medians through the same imputer and scaler as in training. It returns the podium probability and the yes/no decision of the final model (the neural network) at its validation threshold.

Table 15: Inference on a complete and a partially-missing record.

Record	Inputs	P(podium)	Decision
Complete (test row): Verstappen, Saudi Arabian GP 2024	grid 1, `drv_pos` 1, `drv_pts_share` 1.0, `drv_wins` 1	0.688	podium (correct)
Partially missing: a rookie in round 1	grid 4; standings missing	0.259	no podium

14 Limitations

- **Qualifying-coverage drift.** Qualifying times exist only from 1994, Q2/Q3 from 2003, and only for the faster drivers. The model uses the grid (the qualifying result after penalties) and no qualifying times, so information such as the gap to pole is not used.
- **Era sensitivity.** Field sizes, points systems and reliability changed greatly over 75 years. Points are normalised by the leader's points and the split is by season, but training rows from the 1950s still differ strongly from 2022–2024.
- **Dominant teams.** The validation and test seasons are each dominated by one team (Mercedes 2018–2021, Red Bull 2022–2024), so the scores and explanations describe those eras.
- **Race-day events are invisible.** Rain, crashes, safety cars and mechanical failures are not known before the start; the Kvyat example shows the effect.

- **Small validation set.** With 243 validation podiums, differences of about 0.01 PR-AUC between models are within noise; the model ranking is not stable between validation and test.
- **Explanations are not causal** and describe one model on 2018–2021 rows.
- **Q1–Q3** rely on the dataset's status labels and on a simplified nationality-to-country mapping. Mechanical rates include unclear retirements, and the Q2 home definition ignores drivers who live abroad.

15 Conclusion

After auditing and cleaning the 14 tables, a leakage-safe table of 24,724 starts was built and split by season. The data-engineering questions show that front-row starts convert to podiums far more often since 2014 (55.5% $\rightarrow$ 76.0%) with a completely reshuffled circuit ranking, that there is no home advantage once starting position is controlled for, and that mechanical retirements fell from 11.2% to 6.9% of starts. For prediction, the starting grid plus the driver's championship standings before the race are enough: logistic regression, a random forest and a shallow neural network all clearly beat the grid-only baseline (test PR-AUC 0.70–0.72 vs. 0.647) and perform about the same. Ablation and XAI agree that the grid carries most of the signal, with the season standings adding a consistent but smaller contribution.

A Reproducibility and decision log

The notebook runs top to bottom with *Run All*; on Kaggle the CSV path is detected automatically, and the first cell installs `shap`, `lime` and `missingno`. All random seeds are fixed to 42. Every design decision, including the ones superseded during the project, is recorded with its date and evidence in `docs/decisions.md`. The p-values in Table 12 come from a supplementary paired-bootstrap run recorded there (D29).